Scala Coding Challenge Test (2 Hours)

Total Marks: 100

Duration: 120 Minutes

Section	Type	Questions	Marks
Section A	Coding Fundamentals	3	30
Section B	Collections & Functional Programming	3	30
Section C	OOP & Design	2	20
Section D	Debugging	2	20

Section A – Coding Fundamentals (30 Marks)

Question 1 (10 Marks)

Write a Scala program that:

1. Accepts a number **N**
2. Prints all numbers from **1 to N**
3. Prints:

- Sum of numbers
- Average

Example
Input

N = 5

Output

Numbers: 1 2 3 4 5

Sum = 15
Average = 3

```
scala> import scala.io.StdIn
import scala.io.StdIn

scala> var N=StdIn.readInt()
var N: Int = 5

scala> for(i<- 1 to N)
     | println(i)
1
2
3
4
5

scala> var sum=0
var sum: Int = 0

scala> for(i<- 1 to N)
     | sum+=i

scala> println("Sum of NUmbers : "+sum)
Sum of NUmbers : 15

scala> var avg=sum/N
var avg: Int = 3

scala> println("Average : "+avg)
Average : 3
```

Question 2 (10 Marks)

Write a program to check whether a **number is a palindrome**.

Example

Input: 121
Output: Palindrome

```
scala> import scala.io.StdIn
import scala.io.StdIn

scala> def checkPalin(N:Int)={
     | var rev=0
     | var temp=N
     | while(temp>0){
     | rev=rev*10+temp%10
     | temp/=10
     | }
     | if(rev==N)println("Palindrome")
     | else println("Not a Palindrome")
     | }
def checkPalin(N: Int): Unit

scala> val N=StdIn.readInt()
val N: Int = 121

scala> checkPalin(N)
Palindrome
```

Question 3 (10 Marks)

Write a program to generate the **Fibonacci series up to N terms**.

Example

Input: 7
Output:
0 1 1 2 3 5 8

Section B – Collections & Functional Programming (30 Marks)

Question 4 (10 Marks)

Given a list:

val numbers = List(10,20,30,40,50)

Write Scala code to:

1. Multiply each element by **2**
2. Filter numbers **greater than 50**
3. Print the result

Expected Output

List(60,80,100)

```
scala> var numbers=List(10,20,30,40,50)
var numbers: List[Int] = List(10, 20, 30, 40, 50)

scala> numbers=numbers.map(x=>x*2)
// mutated numbers

scala> var result=numbers.filter(x=>x>50)
var result: List[Int] = List(60, 80, 100)

scala> print(result)
List(60, 80, 100)
```

Remarks: Try to reduce the code with some fucnctioins

Question 5 (10 Marks)

Given a sentence:

"scala is powerful and scala is functional"

Write a Scala program to **count occurrences of each word**.

Expected Output

scala -> 2
is -> 2
powerful -> 1
and -> 1
functional -> 1

```
import scala.collection.mutable

scala> val string="scala is powerful and scala is fuctional"
val string: String = scala is powerful and scala is fuctional

scala> val wordMap=mutable.Map[String,Int]()
val wordMap: scala.collection.mutable.Map[String,Int] = HashMap()

scala> var sp=string.split(" ")
var sp: Array[String] = Array(scala, is, powerful, and, scala, is, fuctional)

scala> for(w<-sp){
     | wordMap(w)=wordMap.getOrElse(w,0)+1
     | }

scala> var result=wordMap.toList.sortBy(-_._2)
var result: List[(String, Int)] = List((scala,2), (is,2), (powerful,1), (and,1)
, (fuctional,1))

scala> for(case(word,count)<-result){
     | println(s"$word -> $count")
     | }
scala -> 2
is -> 2
powerful -> 1
and -> 1
fuctional -> 1
```

Remarks: Clarity of the code

Question 6 (10 Marks)

Write a Scala program to **remove duplicates from a list and sort it**.

Example

Input:
List(4,2,1,3,2,4,5)

Output:
List(1,2,3,4,5)

```
scala> var lst=List(4,2,1,3,2,4,5)
var lst: List[Int] = List(4, 2, 1, 3, 2, 4, 5)

scala> lst=lst.distinct.sorted
// mutated lst

scala> print(lst)
List(1, 2, 3, 4, 5)
```

Section C – OOP Concepts (20 Marks)

Question 7 (10 Marks)

Create a **Student class** with:

Attributes:

- id
- name
- marks

Functions:

- display student details
- check if student **passed (marks > 40)**

Example Output

Student: Rahul
Marks: 55
Status: Pass

```
scala> class Student(id:Int,name:String,marks:Int){
     | def details()={
     | println(s"Student : $name")
     | println(s"marks : $marks")
     | println(s"Status : ${if (isPassed()) "Pass" else "Fail"}")
     | }
     | def isPassed()={marks>40}
     | }
class Student

scala> val s1=new Student(1,"Rahul",55)
val s1: Student = Student@7bc02941

scala> s1.details()
Student : Rahul
marks : 55
Status : Pass
```

Remarks: Also try once with auxilary constructor.

Question 8 (10 Marks)

Create a **BankAccount class**.

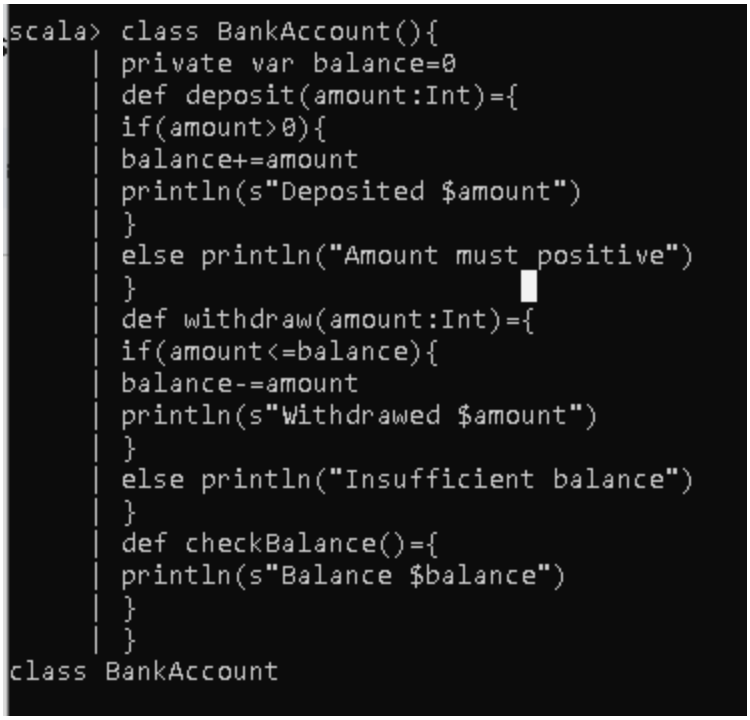
Functions:

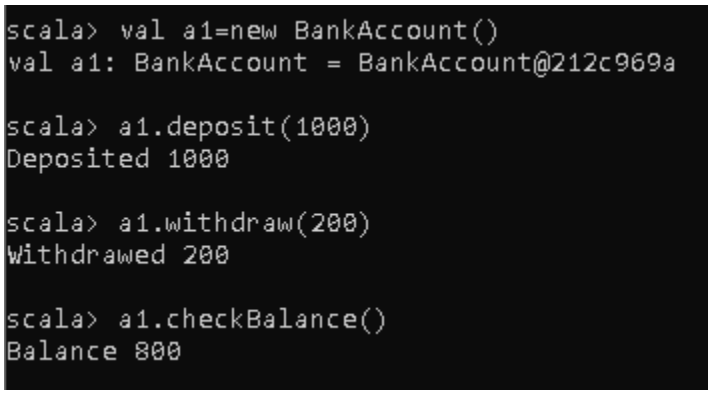
- deposit(amount)
- withdraw(amount)
- checkBalance()

Example

Deposit 1000
Withdraw 200

Balance: 800





Remarks: Add the amount in Double

Section D – Debugging (20 Marks)

Question 9 (10 Marks)

Fix the following code.

val numbers = List(1,2,3,4)

val result = numbers.filter(x => x = 2)

println(result)

Expected Output

List(2)

Fix:

val result = numbers.filter(x => x == 2)

Question 10 (10 Marks)

Identify and fix errors.

```
class Person(name:String){
println("Person name is "+name)
}

val p = new Person
```

Fix:

val p = new Person('jeeva')

Bonus Challenge (Optional – 10 Marks)

Create a **Mini Student Management System**

Features:

- Add student
- Display students
- Search student by name

Use:

- Case class
- List collection
- Functions

```
scala> class StudentManagementSystem{
     | private var students:List[Student]=List()
     | def addStudent(student:Student)={
     | students=students:+student
     | }
     | def searchStudent(name:String)={
     | students.foreach(s=>
     | if(s.name==name){
     | println(s"$name found")
     | println(s"Name : ${s.name}, Age : ${s.age}")
     | })
     | }
     | def displayStudents()={
     | students.foreach(s=>
     | println(s"Name : ${s.name}, Age : ${s.age}"))
     | }
     | }
class StudentManagementSystem
```

```
scala> var s1=new Student("jeeva",25)
var s1: Student = Student(jeeva,25)

scala> var s2=new Student("ravi",20)
var s2: Student = Student(ravi,20)

scala> val sms=new StudentManagementSystem()
val sms: StudentManagementSystem = StudentManagementSystem@53b0fbf6

scala> sms.addStudent(s1)

scala> sms.displayStudents()
Name : jeeva, Age : 25

scala> sms.addStudent(s2)

scala> sms.displayStudents()
Name : jeeva, Age : 25
Name : ravi, Age : 20

scala> sms.searchStudent("jeeva")
jeeva found
Name : jeeva, Age : 25
```

Criteria	Marks
Code correctness	40
Logic implementation	30
Code readability	10
Use of Scala features	10
Efficiency	10